

## Advanced JAVA and J2EE – IV UNIT

**2 marks:**

### 1. What are servlets?

Servlets are small programs that execute on the server side of a web connection. Servlets dynamically extend the functionality of a web server. They can respond to requests from web clients, typically browsers, by generating dynamic content.

### 2. List the drawbacks of CGI programs.

- It was expensive in terms of processor and memory resources to create a separate process for each client request.
- CGI programs were not platform-independent.

### 3. List the advantages of servlet.

- Performance is significantly better. Servlets execute within the address space of a web server.
- Servlets are platform-independent because they are written in Java.
- Java security manager enforces a set of restrictions to protect the resources on a server machine.
- Full functionality of the Java class libraries is available to a servlet.

### 4. With syntax write the purpose of `getParameter()` method.

`String getParameter(String pname)` - Returns the value of the parameter named *pname*.

### 5. What is the purpose of extending the `GenericServlet` class in a servlet, and what methods does it provide by default?

The `GenericServlet` class provides implementations of the basic life cycle methods for a servlet. `GenericServlet` implements the `Servlet` and `ServletConfig` interfaces. In addition, a method to append a string to the server log file is available. The signatures of this method are shown here:

```
void log(String s)
void log(String s, Throwable e)
```

Here, *s* is the string to be appended to the log, and *e* is an exception that occurred.

### 6. Explain the role of the `ServletRequest` and `ServletResponse` objects in the `service()` method.

The `ServletRequest` interface enables a servlet to obtain information about a client request. The `ServletResponse` interface enables a servlet to formulate a response for a client.

### 7. What is `ServletConfig` Interface? Mention any 2 methods.

The `ServletConfig` interface allows a servlet to obtain configuration data when it is loaded. The methods declared by this interface are summarized here:

|                                                 |                                       |
|-------------------------------------------------|---------------------------------------|
| <code>ServletContext getServletContext()</code> | Returns the context for this servlet. |
| <code>String getServletName()</code>            | <code>String getServletName()</code>  |

### 8. What is the purpose and significance of the `getWriter()` method in the context of generating an HTTP response in a servlet?

It returns a `PrintWriter` that can be used to write character data to the response. An `IllegalStateException` is thrown if `getOutputStream()` has been previously invoked on this object.

### 9. \_\_\_\_\_ `Servlet` \_\_\_\_\_ interface declares life cycle methods for a servlet and \_\_\_\_\_ `ServletConfig` \_\_\_\_\_ interface allows servlets to get initialization parameters.

### 10. What is the role of the `getServletConfig()` and `getServletInfo()` methods in the `Servlet` interface.

The `getServletConfig()` method is called by the servlet to obtain initialization parameters. A servlet developer overrides the `getServletInfo()` method to provide a string with useful information (for example, author, version, date, copyright). This method is also invoked by the server.

**11. Differentiate between the `getInitParameter(String param)` and `getInitParameterNames()` methods in the `ServletConfig` interface.**

|                                                                |                                                                        |
|----------------------------------------------------------------|------------------------------------------------------------------------|
| <code>String getInitParameter(String param)</code>             | Returns the value of the initialization parameter named <i>param</i> . |
| <code>Enumeration&lt;String&gt; getInitParameterNames()</code> | Returns an enumeration of all initialization parameter names.          |

**12. What is the purpose and usage of the `getAttribute(String attr)` and `setAttribute(String attr, Object val)` methods in the `ServletContext` interface.**

|                                                         |                                                                                 |
|---------------------------------------------------------|---------------------------------------------------------------------------------|
| <code>Object getAttribute(String attr)</code>           | Returns the value of the server attribute named <i>attr</i> .                   |
| <code>void setAttribute(String attr, Object val)</code> | Sets the attribute specified by <i>attr</i> to the value passed in <i>val</i> . |

**13. What is the purpose of the `getParameterNames()` and `getParameterValues(String name)` methods in the `ServletRequest` interface.**

|                                                       |                                                                                             |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>Enumeration getParameterNames()</code>          | Returns an enumeration of the parameter names for this request.                             |
| <code>String[] getParameterValues(String name)</code> | Returns an array containing values associated with the parameter specified by <i>name</i> . |

**14. What is the usage of `readLine(byte[] buffer, int offset, int size)` method in the `ServletInputStream` class?**

The `readLine()` method is provided to read bytes from the stream. It is shown here:

`int readLine(byte[] buffer, int offset, int size)` throws `IOException`

Here, `buffer` is the array into which size bytes are placed starting at `offset`. The method returns the actual number of bytes read or `-1` if an end-of-stream condition is encountered.

**15. What's the specific purpose of `ServletOutputStream` and `ServletInputStream`?**

The `ServletInputStream` class extends `InputStream`. It is implemented by the servlet container and provides an input stream that a servlet developer can use to read the data from a client request.

The `ServletOutputStream` class extends `OutputStream`. It is implemented by the servlet container and provides an output stream that a servlet developer can use to write data to a client response.

**16. List classes that are provided in the `javax.servlet.http` package.**

|                                      |                                                                        |
|--------------------------------------|------------------------------------------------------------------------|
| <code>Cookie</code>                  | Allows state information to be stored on a client machine.             |
| <code>HttpServlet</code>             | Provides methods to handle HTTP requests and responses.                |
| <code>HttpSessionEvent</code>        | Encapsulates a session-changed event.                                  |
| <code>HttpSessionBindingEvent</code> | Indicates when a listener is bound to or unbound from a session value. |

**17. Write the usage of any two methods described in the `HttpServletResponse` interface.**

|                                                   |                                                                                       |
|---------------------------------------------------|---------------------------------------------------------------------------------------|
| <code>void addCookie(Cookie cookie)</code>        | Adds <i>cookie</i> to the HTTP response.                                              |
| <code>boolean containsHeader(String field)</code> | Returns <b>true</b> if the HTTP response header contains a field named <i>field</i> . |

**18. What is Cookie? How it is helpful?**

The **Cookie** class encapsulates a cookie. A *cookie* is stored on a client and contains state information. Cookies are valuable for tracking user activities. For example, assume that a user visits an online store. A cookie can save the user's name, address, and other information. The user does not need to enter this data each time he or she visits the store.

**19. Describe the significance of the valueBound and valueUnbound methods in the HttpSessionBindingListener interface.**

valueBound() - This method is called whenever an object is bound to an HTTP session, i.e, added to an HTTP session.

valueUnbound() -This method is called whenever an object is unbound from an HTTP session, i.e, removed from an HTTP session.

**20. What information can be stored in a cookie?**

Some of the information that can be saved for each cookie includes the following:

- The name of the cookie
- The value of the cookie
- The expiration date of the cookie
- The domain and path of the cookie

**21. What is the significance of the getSession() method of HttpServletRequest.**

A session can be created via the getSession() method of HttpServletRequest. An HttpSession object is returned. This object can store a set of bindings that associate names with objects. The setAttribute(), getAttribute(), getAttributeNames(), and removeAttribute() methods of HttpSession manage these bindings. Session state is shared by all servlets that are associated with a client.

**22. Write the purpose of next() and getString().**

The next() method of the ResultSet moves the cursor forward one row from its current position

The getString() method of the ResultSet object is used to copy the value of a specified column in the current row of the ResultSet to a String object.

**23. List the parts of URL that is used in getConnection() method to establish connection.**

- jdbc, which indicates that the JDBC protocol is to be used to read the URL
- <subprotocol>, which is the JDBC driver name
- <subname>, which is the name of the database

**24. Write the purpose of setLoginTimeout() and getLoginTimeout().**

ofsetLoginTimeout() can be used by the J2EE component to establish the maximum time the DriverManager waits for a response from a DBMS before timing out.

getLoginTimeout() is used to retrieve from the DriverManager the maximum time the DriverManager is set to wait until it times out. It returns an int that represents seconds.

**25. Write the purpose of forName() and createStatement().**

The Class.forName() method is used to load the JDBC driver. The name of the driver is passed to it. Example:     Class.forName( "sun.jdbc.odbc.JdbcOdbcDriver");

Connect.createStatement() is used to create a Statement object. The Statement object is then used to execute a query and return a ResultSet object that contains the response from the DBMS.

Example:     Statement stcon.createStatement();

**26. What is the main purpose of the JDBC to ODBC (Type 1) driver?**

The main purpose of the JDBC to ODBC (Type 1) driver is to provide a bridge that allows Java applications to connect to databases that support the Open Database Connectivity (ODBC) standard. This driver acts as an intermediary between JDBC and ODBC, translating JDBC calls into ODBC calls, thereby enabling the use of ODBC drivers to access databases.

**27. Differentiate between Type 3 and Type 4 JDBC drivers. Which one is considered to be the fastest way to communicate SQL queries to the DBMS?**

| Type 3 Driver (Java Protocol)                                                 | Type 4 Driver (Database Protocol)      |
|-------------------------------------------------------------------------------|----------------------------------------|
| Uses a middleware server                                                      | Connects directly to the database      |
| Converts SQL to JDBC format first, then into the format required by the DBMS. | Translates SQL directly to DBMS format |
| Generally slower                                                              | Generally faster                       |

**28. What is the difference between the executeQuery() and executeUpdate() methods of a Statement object in JDBC?**

executeQuery(): This method returns the ResultSet object that contains rows, columns and metadata that represent data requested by the query. This method is used to execute only the 'SELECT' query of SQL.

executeUpdate(): This method is used to execute the queries that contain INSERT, DELETE and UPDATE statements. It returns integer indicating the number of rows that were updated by the query.

**29. Briefly explain what a ResultSet object represents in JDBC.**

ResultSet object contains data requested by the query. It also contains methods that are used to copy data from the ResultSet into a Java collection of objects or variable(s) for further processing. Data in a ResultSet object is logically organized into a virtual table consisting of rows and columns.

**30. What is the main advantage of using a PreparedStatement object over a Statement object in JDBC?**

PreparedStatement objects are pre-compiled by the database when they are first created. When you execute the same PreparedStatement multiple times with different data, the database can reuse this plan, leading to faster execution compared to Statement objects where the query is recompiled each time.

**31. How do PreparedStatement objects handle dynamic values in SQL queries?**

PreparedStatement objects handle dynamic values in SQL queries by using placeholders (also known as parameter markers) denoted by question marks (?) within the SQL statement. These placeholders are then substituted with actual values using setter methods provided by the PreparedStatement class. This approach ensures that the SQL statement remains constant while allowing the values to change dynamically.

**32. What are the different parameters used by CallableStatement object?**

|       |                                                                                                                                                     |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| IN    | A parameter whose value is unknown when the SQL statement is created. You bind values to IN parameters with the setXXX() methods.                   |
| OUT   | A parameter whose value is supplied by the SQL statement it returns. You retrieve values from the OUT parameters with the getXXX() methods.         |
| INOUT | A parameter that provides both input and output values. You bind variables with the setXXX() methods and retrieve values with the getXXX() methods. |

**33. How to Insert a Row in to the ResultSet?**

The updateXXX() method is used to specify the column and value that will be placed into the column of the ResultSet. The updateXXX() method requires two parameters, one to specify either the name or the number of the column and the other to specify the new value. The insertRow() method is called after the updateXXX() methods, which causes a new row to be inserted into the ResultSet having values of the parameters in the updateXXX() methods.

### 34. Why Savepoints are used in database transaction?

Savepoints in database transactions are used primarily to provide a way to roll back parts of a transaction rather than the entire transaction. It is a virtual marker that defines the task at which the rollback stops.

### 35. How to batch sql statement into transaction statement?

The `addBatch()` method of the `Statement` object is used to batch together sql statement into a single transaction statement. It receives a SQL statement as a parameter and places the SQL statement in the batch.

### 36. What are the methods supported by RowSetListener class?

- **`rowSetChanged(RowSetEvent event)`**: This method is called when the data in the `RowSet` object has changed, typically because the `RowSet` has been re-executed or its data has been modified while connected to the database.
- **`rowChanged(RowSetEvent event)`**: This method is called when a row in the `RowSet` object has been changed, indicating that one or more column values in the row have been modified.
- **`cursorMoved (RowSetEvent event)`**: This method is called when the cursor position within the `RowSet` object has been moved, indicating that the current row has changed.

### 37. What is JavaServerPages?

Java Server Pages (JSP) is a technology for developing Web pages that supports dynamic content. This helps developers insert java code in HTML pages by making use of special JSP tags, most of which start with `<%` and end with `%>`. A Java Server Pages component is a type of Java servlet that is designed to fulfill the role of a user interface for a Java web application.

### 38. List any four advantages of using JSP.

- Performance is significantly better.
- JSP are always compiled before they are processed by the server.
- Java Server Pages are built on top of the Java Servlets API.
- JSP pages can be used in combination with servlets that handle the business logic.

### 39. List any four implicit Objects.

- `request` - This is the `HttpServletRequest` object associated with the request.
- `response` - This is the `HttpServletResponse` object associated with the response to the client.
- `out` - This is the `PrintWriter` object used to send output to the client.
- `session` - This is the `HttpSession` object associated with the request.

### 40. What is the usage of buffer attribute in JSP?

The `buffer` attribute specifies the buffering characteristics for the server output response object. You may code a value of "none" to specify no buffering so that the servlet output is immediately directed to the response object or you may code a maximum buffer size in kilobytes, which directs the servlet to write to the buffer before writing to the response object.

### 41. What is page Directive?

The page directive is used to provide instructions to the container. These instructions pertain to the current JSP page. You may code page directives anywhere in your JSP page. By convention, page directives are coded at the top of the JSP page. Following is the basic syntax of the page directive:  
`<%@ page attribute="value" %>`

## *Long Answer Questions*

### 1. Explain the three key methods in the lifecycle of a servlet (`init()`, `service()`, `destroy()`).

- `init()` method is called **only once** when the servlet is first loaded into the memory. It is possible to pass initialization parameters to the servlet so it may configure itself.

ii. service() method is called to process the HTTP request. You will see that it is possible for the servlet to read data that has been provided in the HTTP request. It may also formulate an HTTP response for the client. The servlet remains in the server's address space and is available to process any other HTTP requests received from clients. The service() method is called for each HTTP request.

iii. destroy() method is called **only once** just before the servlet is unloaded from the memory. It's used to perform any cleanup tasks required by the servlet, such as closing database connections, releasing resources acquired during initialization, flushing buffers etc.

## 2. Develop a basic servlet program that displays a welcome message to the user.

### HelloServlet.java

```
import java.io.*;
import javax.servlet.*;
public class HelloServlet extends GenericServlet {
    public void service(ServletRequest request, ServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter pw = response.getWriter();
        pw.println("Welcome!");
        pw.close();
    }
}
```

### web.xml

```
<servlet>
    <servlet-name>HelloServlet</servlet-name>
    <servlet-class>HelloServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>HelloServlet</servlet-name>
    <url-pattern>/servlet/HelloServlet</url-pattern>
</servlet-mapping>
```

## 3. Explain the functionalities of the following methods available in ServletConfig Interface.

| Method                                        | Description                                                        |
|-----------------------------------------------|--------------------------------------------------------------------|
| ServletContext getServletContext()            | Returns the context for this servlet.                              |
| String getInitParameter(String <i>param</i> ) | Returns value of the initialization parameter named <i>param</i> . |
| Enumeration<String> getInitParameterNames()   | Returns an enumeration of all initialization parameter names.      |
| String getServletName()                       | Returns the name of the invoking servlet.                          |

## 4. Write the usage of any four methods of ServletRequest interface.

|                                                  |                                                                                             |
|--------------------------------------------------|---------------------------------------------------------------------------------------------|
| Object getAttribute(String <i>attr</i> )         | Returns the value of the attribute named <i>attr</i> .                                      |
| String getCharacterEncoding()                    | Returns the character encoding of the request.                                              |
| Enumeration getParameterNames()                  | Returns an enumeration of the parameter names for this request.                             |
| String[] getParameterValues(String <i>name</i> ) | Returns an array containing values associated with the parameter specified by <i>name</i> . |

## 5. Write the usage of any four methods of ServletResponse interface.

|                                                |                                                                                                                                                                                      |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| String getCharacterEncoding( )                 | Returns the character encoding for the response.                                                                                                                                     |
| PrintWriter getWriter( )<br>throws IOException | Returns a PrintWriter that can be used to write character data to the response. An IllegalStateException is thrown if getOutputStream( ) has been previously invoked on this object. |
| void setContentLength(int size)                | Sets the content length for the response to <i>size</i> .                                                                                                                            |
| void setContentType(String type)               | Sets the content type for the response to <i>type</i> .                                                                                                                              |

**6. Write a Java servlet named FormProcessorServlet that can handle HTTP POST requests containing form data. The servlet should: Read Parameters: Extract the names and values of parameters submitted from the form. Process Data: Access the specific parameter values for fields like "name" and "email" (assuming those are the form field names). Generate Response: Create an HTML response that displays the submitted data back to the user in a user-friendly format.**

### Index.html

```
<html>    <head><title>Form</title></head>
<body> <center>
<form method="post" action="http://localhost:8081/sample/a">
<table> <tr> <td> <b>Employee</b>
        <td><input type="text" name="e" size="25" value=""></td> </tr>
        <tr> <td> <b>Email</b>
        <td><input type="email" name="p" size="50" value=""></td> </tr>
</table>
<input type="submit" value="Submit">
</form> </center> </body> </html>
```

### Web.xml

```
<web-app> <servlet>
    <servlet-name>FormProcessorServlet</servlet-name>
    <servlet-class>Form ProcessorServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>FormProcessorServlet</servlet-name>
    <url-pattern>/a</url-pattern>
</servlet-mapping> </web-app>
```

### formProcessorServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class Form ProcessorServlet extends HttpServlet {
    public void doPost(HttpServletRequest request, HttpServletResponse response) throws
        ServletException, IOException {
        String name = request.getParameter("e");
        String email = request.getParameter("p");
        response.setContentType("text/html");
        Print Writer pw = response.getWriter(); pw.println("<html><body>");
        pw.println("<b>Name of the Employee is: </b>" + name + "<br>");
        pw.println("<b>Email of the Employee is: </b>" + email);
        pw.println("</body></html>");
        pw.close();
    }
}
```

**7. Explain the following methods of HttpServletRequest interface with their syntax**  
**i. get\_cookies   ii. getMethod   iii. getPathInfo   iv. getSession**

|                                        |                                                                                                                                                               |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cookie[] get_cookies( )                | Returns an array of the cookies in this request.                                                                                                              |
| String getMethod( )                    | Returns the HTTP method for this request.                                                                                                                     |
| String getPathInfo( )                  | Returns any path information that is located after the servlet path and before a query string of the URL.                                                     |
| HttpSession getSession( )              | Returns the session for this request. If a session does not exist, one is created and then returned.                                                          |
| HttpSession<br>getSession(boolean new) | If <i>new</i> is <b>true</b> and no session exists, creates and returns a session for this request. Otherwise, returns the existing session for this request. |

**8. With an example, Explain the purpose and behavior of the doGet() method in a servlet class.**

The doGet( ) method is overridden to process any HTTP GET requests that are sent to this servlet. It uses the getParameter( ) method of HttpServletRequest to obtain the selection that was made by the user. A response is then formulated.

**index.html**

```
<html> <body>   <center>
<form action="http://localhost:8080/servlets-examples/servlet/ColorGetServlet">
    <b>Color:</b>
    <select name="color" size="1">
        <option value="Red">Red</option>
        <option value="Green">Green</option>
        <option value="Blue">Blue</option>
    </select>   <br><br>
    <input type=submit value="Submit">
</form> </body> </html>
```

**ColorGetServlet.java**

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class ColorGetServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String color = request.getParameter("color");
        response.setContentType("text/html");
        PrintWriter pw = response.getWriter();
        pw.println("<b>The selected color is: ");
        pw.println(color);
        pw.close();
    } }
}
```

**9. With an example, Explain the purpose and behavior of the doPost() method in a servlet class.**

The doPost( ) method is overridden to process any HTTP POST requests that are sent to this servlet. It uses the getParameter( ) method of HttpServletRequest to obtain the selection that was made by the user. A response is then formulated.

## index.html

```
<html> <body> <center>
<form method="post" action="http://localhost:8080/servlets-examples/servlet/ColorPostServlet">
    <b>Color:</b>
    <select name="color" size="1">
        <option value="Red">Red</option>
        <option value="Green">Green</option>
        <option value="Blue">Blue</option>
    </select> <br><br>
    <input type="submit" value="Submit">
</form> </body> </html>
```

## ColorPostServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class ColorPostServlet extends HttpServlet {
    public void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String color = request.getParameter("color"); //Get the selected color from the requested parameter
        response.setContentType("text/html"); //Set the response content type
        PrintWriter pw = response.getWriter(); //Get PrintWriter object
        pw.println("<b>The selected color is: "); //Write HTML response
        pw.println(color);
        pw.close(); //Close PrintWriter
    }
}
```

## **10. What are the different methods involved in the process of session management in servlets? Explain their purpose.**

***getSession(boolean create)***: This method retrieves or creates an HttpSession object associated with the current request. If no session exists and *create* is set to *true*, a new session will be created. If *create* is set to *false* and no session exists, *null* is returned.

Example: HttpSession session = request.getSession(true); // create a session if none exists

***setAttribute(String name, Object value)***: Binds an object to a specific name in the session. The *name* parameter specifies the name under which the object will be stored and *value* indicates the object to be associated with the name.

***getAttribute(String name)***: Retrieves an object associated with a given name from the session. The *name* parameter specifies the name under which the object was bound to the session. It returns the object bound to the specified name, or null if no object is associated with that name.

***removeAttribute(String name)***: Removes an object associated with a given name from the session. The *name* parameter specifies the name of the object to be removed. It returns None(void method).

***getAttributeNames()***: This method retrieves an enumeration of all attribute names currently bound to the session. It returns an Enumeration object containing the names of all attributes in the session.

## **11. How cookies can be handled using servlet.**

The Cookie class encapsulates a cookie. A cookie is stored on a client and contains state information. Cookies are valuable for tracking user activities. For example, assume that a user visits an online store. A cookie can save the user's name, address, and other information. The user does not need to enter this data each time he or she visits the store.

A servlet can write a cookie to a user's machine via the `addCookie()` method of the `HttpServletResponse` interface. The data for that cookie is then included in the header of the HTTP response that is sent to the browser. The names and values of cookies are stored on the user's machine. Some of the information that is saved for each cookie includes the following:

- The name of the cookie
- The value of the cookie
- The expiration date of the cookie
- The domain and path of the cookie

The expiration date determines when this cookie is deleted from the user's machine. If an expiration date is not explicitly assigned to a cookie, it is deleted when the current browser session ends. Otherwise, the cookie is saved in a file on the user's machine.

The domain and path of the cookie determine when it is included in the header of an HTTP request. If the user enters a URL whose domain and path match these values, the cookie is then supplied to the Web server. Otherwise, it is not.

There is one constructor for `Cookie`. It has the signature shown here:

`Cookie(String name, String value)`

Here, the name and value of the cookie are supplied as arguments to the constructor.

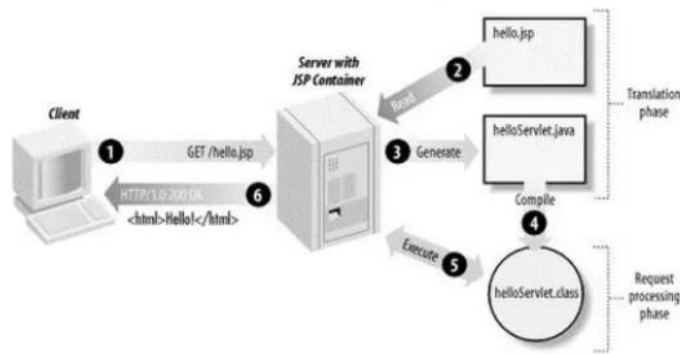
## **12. Explain why JSP is a compelling choice for web development compared to the Common Gateway Interface (CGI).**

Java Server Pages often serve the same purpose as programs implemented using the Common Gateway Interface (CGI). But JSP offers several advantages in comparison with the CGI.

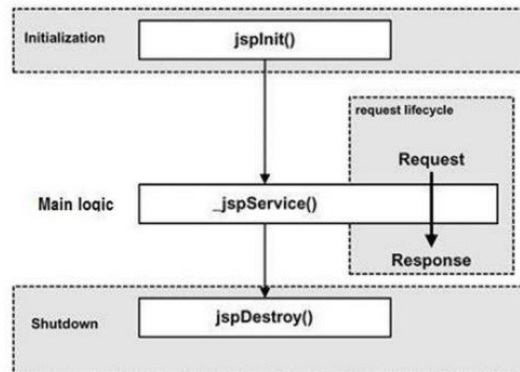
- Performance is significantly better because JSP allows embedding Dynamic Elements in HTML Pages itself instead of having separate CGI files.
- JSP are always compiled before they are processed by the server unlike CGI/Perl which requires the server to load an interpreter and the target script each time the page is requested.
- Java Server Pages are built on top of the Java Servlets API, so like Servlets, JSP also has access to all the powerful Enterprise Java APIs, including JDBC, JNDI, EJB, JAXP, etc.
- JSP pages can be used in combination with servlets that handle the business logic, the model supported by Java servlet template engines.

## **13. Explain the key steps involved in how a web server processes a JSP request and generates a web page.**

- As with a normal page, browser (client) sends an HTTP request to the web server.
- The web server recognizes that the HTTP request is for a JSP page and forwards it to a JSP engine. This is done by using the URL or JSP page which ends with `.jsp` instead of `.html`.
- The JSP engine loads the JSP page from disk and converts it into a servlet content. This conversion is very simple in which all template text is converted to `println()` statements and all JSP elements are converted to Java code. This code implements the corresponding dynamic behavior of the page.
- The JSP engine compiles the servlet into an executable class and forwards the original request to a servlet engine.
- Apart of the web server called the servlet engine loads the Servlet class and executes it. During execution, the servlet produces an output in HTML format. This output is further passed on to the web server by the servlet engine inside an HTTP response.
- The web server forwards the HTTP response to our browser in terms of static HTML content.
- Finally, the web browser handles the dynamically-generated HTML page inside the HTTP response exactly as if it were a static page.



#### 14. Explain the key stages involved in the lifecycle of a Java Server Page (JSP).



#### JSP Compilation

When a browser asks for a JSP, the JSP engine first checks to see whether it needs to compile the page. If the page has never been compiled, or if the JSP has been modified since it was last compiled, the JSP engine compiles the page. The compilation process involves three steps:

- Parsing the JSP.
- Turning the JSP in to a servlet.
- Compiling the servlet.

#### JSP Initialization

When a container loads a JSP it invokes the `jspInit()` method before servicing any requests. Typically, initialization is performed only once we generally initialize database connections, open files, and create lookup tables in the `jspInit` method. If we need to perform JSP-specific initialization, override the `jspInit()` method:

```
Public void jspInit(){           //Initializationcode... }
```

#### JSP Execution

This phase of the JSP life cycle represents all interactions with requests until the JSP is destroyed. Whenever a browser requests a JSP and the page has been loaded and initialized, the JSP engine invokes the `_jspService()` method in the JSP.

The `_jspService()` method of a JSP is invoked on request basis. This is responsible for generating the response for that request and this method is also responsible for generating responses to all seven of the HTTP methods, i.e., GET, POST, DELETE, etc.

#### JSP Cleanup

The destruction phase of the JSP lifecycle represents when a JSP is being removed from use by a container. The `jspDestroy()` method is the destroy method. Override `jspDestroy` when you need to perform any cleanup, such as releasing database connections or closing open files. The `jspDestroy()` method has the following form:

```
public void jspDestroy() { //Yourcleanupcodegoeshere. }
```

## 15. Write short note on JSP directives and JSP Actions.

### JSP Directives

A JSP directive affects the overall structure of the servlet class. It usually has the following form:

```
<%@directive attribute="value"%>
```

There are three types of directive tag:

| Directive         | Description                                                                                           |
|-------------------|-------------------------------------------------------------------------------------------------------|
| <%@ page... %>    | Defines page-dependent attributes, such as scripting language, error page and buffering requirements. |
| <%@ include... %> | Includes a file during the translation phase.                                                         |
| <%@ taglib... %>  | Declares a tag library, containing custom actions, used in the page                                   |

### JSP Actions

JSP actions use constructs in XML syntax to control the behavior of the servlet engine. You can dynamically insert a file, reuse JavaBeans components, forward the user to another page, or generate HTML for the Java plugin. There is only one syntax for the Action element, as it conforms to the XML standard:

```
<jsp:action_name attribute="value"/>
```

Action elements are basically predefined functions. Following table lists out the JSP Actions:

| Syntax          | Purpose                                             |
|-----------------|-----------------------------------------------------|
| jsp:include     | Includes a file at the time the page is requested.  |
| jsp:useBean     | Finds or instantiates a JavaBean.                   |
| jsp:setProperty | Sets the property of a JavaBean.                    |
| jsp:getProperty | Inserts the property of a JavaBean into the output. |

## 16. Explain any four buffer attributes with examples.

The buffer attribute specifies the buffering characteristics for the server output response object.

You may code a value of "none" to specify no buffering so that the servlet output is immediately directed to the response object or you may code a maximum buffer size in kilobytes, which directs the servlet to write to the buffer before writing to the response object.

To direct the servlet to write the output directly to the response output object, use the following:

```
<%@ page buffer="none" %>
```

Use the following to direct the servlet to write the output to a buffer of size not less than 8 kilobytes:

```
<%@ page buffer="8kb" %>
```

## 17. Explain the steps in the JDBC process.

### Step 1: Loading the JDBC Driver

The JDBC driver must be loaded before the J2EE component can connect to the DBMS. The **Class.forName()** method is used to load the JDBC driver.

The developer must write a routine that loads the JDBC / ODBC Bridge driver called `sun.jdbc.odbc.JdbcOdbcDriver`. The driver is loaded by calling the `Class.forName()` method and passing it the name of the driver, as shown in the following code segment

```
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
```

## Step 2: Connect to the DBMS

Once the driver is loaded, the J2EE component must connect to the DBMS using the `DriverManager.getConnection()` method. This method is used to pass the URL of the database, along with the user ID and password if required by the DBMS. It returns a `Connection` interface that is used throughout the process to reference the database.

Eg: `Connection con = DriverManager.getConnection(url, username, password);`

## Step 3: Create and Execute an SQL Statement

The `Connect.createStatement()` is used to create a `Statement` object. The `Statement` object is then used to execute a query and return a `ResultSet` object that contains the response from the DBMS, which is usually one or more rows of information requested by the J2EE application. Typically, the query is assigned to a `String` object, which is passed to the `Statement` object's `executeQuery()` method.

Eg: `Statement s = connection.createStatement();`  
`ResultSet rs = s.executeQuery("SELECT * FROM employees");`

## Step 4: Process Data Returned by the DBMS

The `java.sql.ResultSet` object is assigned the results received from the DBMS after the query is processed. The `java.sql.ResultSet` object consists of methods used to interact with data that is returned by the DBMS to the J2EE application.

## Step 5: Terminate the Connection to the DBMS

The connection to the DBMS is terminated by using the `close()` method of the `Connection` object once the J2EE application is finished accessing the DBMS. It releases the databases and JDBC resources immediately.

Eg: `rs.close();`  
`s.close();`  
`con.close();`

## **18. Write the steps to associate database with JDBC/ODBC bridge.**

1. Select Start | Settings, and then the Control Panel.
2. Select ODBC 32 to display the ODBC Data Source Administrator.
3. Add a new user by selecting the Add button.
4. Select the driver then select Finish. Use the Microsoft Access Driver if you are using Microsoft Access; otherwise, select the driver for the DBMS that you are using. If you don't find the driver for your DBMS on the list, you'll need to install the driver. Contact the manufacturer of the DBMS for more information on how to obtain the driver.
5. Enter the name of the database as the Data Source name in the ODBC Microsoft Access Setup dialog box. This is the name that will be used within your Java database program to connect to the DBMS.
6. Enter a description for the data source. This is optional, but will be a reminder of the kind of data that is stored in the database.
7. Click the Select button: You'll be prompted to browse the directory of each hard drive connected to your computer in order to define the direct path to the database. Click OK once you locate the database, and the directory path and the name of the database will be displayed in the ODBC Microsoft Access Setup dialog box.
8. Since this is your database, you can determine if a login name and password is required to access the database.
9. If so, then click the Advanced button to display the Set Advanced Options dialog box. This dialog box is used to assign a login name, also referred to as a user ID, and a password to the database. Select OK. If not, then skip this step.
10. When the ODBC Microsoft Access Setup dialog box appears, select OK.
11. Select OK to close the ODBC Data Source Administrator dialog box

## **19. What is JDBC Driver? Explain different types.**

JDBC Drivers: JDBC uses drivers to translate generalized JDBC calls into vendor-specific database calls. Drivers exist for most popular databases. Four Classes of JDBC drivers exist.

### **Type 1 JDBC to ODBC Driver**

A common Type 1 driver defines a JDBC to ODBC bridge. ODBC is the database connectivity for databases. JDBC driver translates JDBC calls to corresponding ODBC calls.

Thus if ODBC driver exists for a database this bridge can be used to communicate with the database from a Java application.

### **Type 2 Java/Native Code Driver**

Type 2 driver communicates directly with native API. It is more efficient since there is one less layer to contend with (i.e. no ODBC). It is dependent on the existence of a native API for a database

### **Type 3 JDBC Driver**

The Type 3 JDBC driver, also referred to as the Java Protocol, is the most commonly used JDBC driver. The Type 3 JDBC driver converts SQL queries into JDBC-formatted statements. The JDBC-formatted statements are translated into the format required by the DBMS.

### **Type 4 JDBC Driver**

The Type 4 JDBC driver is also known as the Type 4 Database Protocol. This driver is similar to the Type 3 JDBC driver, except SQL queries are translated into the format required by the DBMS. SQL queries do not need to be converted to JDBC-formatted systems. This is the fastest way to communicate SQL queries to the DBMS.

## **20. Explain the steps to connect to the database in java?**

- **Choose a Database:** First, you need to choose the database you want to connect to. Common options include MySQL, PostgreSQL, Oracle, SQLite, etc.
- **Download the JDBC Driver:** Download the appropriate JDBC driver for your chosen database and include it in your project's classpath.
- **Load the Driver:** Use `Class.forName()` to load the JDBC driver class. This step is necessary to register the drive with the `DriverManager`.
- **Establish Connection:** Use the `DriverManager.getConnection()` method to establish a connection to the database. You need to provide the URL of the database, username, and password as parameters.
- **Create a Statement:** Once the connection is established create a `Statement` object. Statements are used to execute SQL queries.
- **Execute SQL Queries:** Use the `executeQuery()` method to execute `SELECT` queries or `executeUpdate()` method for `INSERT`, `UPDATE`, `DELETE` queries.
- **Process Results:** If you've executed a `SELECT` query, you can process the results returned by the database. Use methods like `ResultSet.next()` to iterate through the result set and retrieve data.
- **Close Resources:** After you're done with the database operations, make sure to close the `ResultSet`, `Statement`, and `Connection` objects to release the database and JDBC resources.

## **21. What are the JDBC statements? Explain.**

In JDBC (Java Database Connectivity), statements are objects used to execute SQL queries against a database. There are mainly three types of statements in JDBC:

1. **Statement:** This interface is used to execute simple SQL queries without parameters. It can be used for executing queries such as SELECT, INSERT, UPDATE, DELETE, and DDL (Data Definition Language) statements.

```
Statement s = con.createStatement();  
ResultSet rs = s.executeQuery("SELECT * FROM my_table");
```

2. **PreparedStatement:** This interface extends Statement and provides additional methods for executing parameterized SQL queries. PreparedStatement is precompiled, which can improve performance when executing the same SQL statement multiple times with different parameters.

```
PreparedStatement p = con.prepareStatement("INSERT INTO table (col1, col2) VALUES (?, ?);");  
s.setString(1, "value1");  
s.setString(2, "value2");  
int rowsAffected = p.executeUpdate();
```

3. **CallableStatement:** This interface is used to execute stored procedures or functions in the database. Callable Statement allows you to call database stored procedures or functions with IN, OUT, and INOUT parameters.

```
CallableStatement c = con.prepareCall("{call my_stored_procedure(?, ?);");  
c.setInt(1, parameter1);  
c.registerOutParameter(2, Types.INTEGER);  
c.execute();
```

## 22. Describe the following i) Scrollable ResultSet ii) Updatable ResultSet

**i. Scrollable ResultSet:** It is a type of ResultSet that allows you to move the cursor forward and backward, as well as to move to a specific row or to a relative position from the current row. By default, ResultSets in JDBC are forward-only, meaning you can only traverse them in one direction forward from the first row to the last. However, if you need more flexibility in navigating the result set, you can create a scrollable result set. Six methods of the ResultSet object are used to position the virtual cursor, in addition to the next() method.

The **first()** method moves the virtual cursor to the first row in the ResultSet.

The **last()** method positions the virtual cursor at the last row in the ResultSet.

The **previous()** method moves the virtual cursor to the previous row.

The **absolute()** method positions the virtual cursor at the row number specified by the integer passed as a parameter to The absolute() method.

The **relative()** method moves the virtual cursor the specified number of rows contained in the parameter.

The **getRow()** method returns an integer that represents the number of the current row in the ResultSet.

**ii. Updatable ResultSet:** An updatable ResultSet is a feature provided by JDBC that allows you to update the rows retrieved from a database table through a ResultSet object.

ResultSet concurrency determines whether the ResultSet can be updated, or only read. Not all databases ResultSet is updated and JDBC drivers support that the ResultSet is updated.

ResultSet can have one of two concurrency levels

CONCUR\_READ\_ONLY means the ResultSet can only be read

CONCUR\_UPDATABLE means that the ResultSet can be both read and updated

### 23. Write a note on DatabaseMetaData interface.

The DatabaseMetaData interface is used to retrieve information about databases, tables, columns, and indexes among other information about the DBMS.

AJ2EE component retrieves metadata about the database by calling the **getMetaData()** method of the Connection object. The getMetaData() method returns a DatabaseMetaData object that contains information about the database and its components. Once the DatabaseMetaData object is obtained, an assortment of methods contained in the DatabaseMetaData object are called to retrieve specific metadata. Here are some of the more commonly used DatabaseMetaData object methods:

- **getDatabaseProductName()** Returns the product name of the database.
- **getUserName()** Returns the username.
- **getURL()** Returns the URL of the database
- **getSchemas()** Returns all the schema names available in this database.
- **getPrimaryKeys()** Returns primary keys.
- **getProcedures()** Returns stored procedure names.
- **getTables()** Returns names of tables in the database.

### 24. Explain the types of Exceptions occur in JDBC.

There are three kinds of exceptions that are thrown by JDBC methods. These are SQLExceptions, SQLWarnings, and DataTruncation.

- SQLExceptions commonly reflect a SQL syntax error in the query and are thrown by many of the methods contained in the java.sql package. Hopefully, the syntax errors in your code get resolved quickly. In production, this exception is most commonly caused by connectivity issues with the database. It can also be caused by subtle coding errors like trying to access an object that's been closed.

For example, you try to roll back a transaction in a catch clause and don't check first if the database connection is still valid. The getNextException() method of the SQLExceptions object is used to return details about the SQL error or a null if the last exception was retrieved. The getErrorCode() method of the SQLException object is used to retrieve vendor-specific error codes.

- The SQLWarning throws warnings received by the Connection from the DBMS. The getWarnings() method of the Connection object retrieves the warning and the getNextWarning() method of the Connection object retrieves subsequent warnings.
- Whenever data is lost due to truncation of the data value, a DataTruncation exception is thrown.